Heterogeneous Multi-Agent Task-Assignment with Uncertain Execution Times and Preferences

Qinshuang Wei, Vaibhav Srivastava, Vijay Gupta

Abstract— **While task assignment problems for a single agent have been widely studied, multi-agent task assignment problems with heterogeneous task preferences or characteristics remain relatively less well-characterized. We study a multi-agent task assignment problem where a central planner needs to assign tasks to multiple members of a team. The members have heterogeneous capabilities in terms of task completion times and preferences in terms of rewards they collect upon task completion. We assume that both the reward and execution time for each member to complete any task follow unknown stochastic distributions. The goal of the assigner is to maximize the total expected reward the team receives over the planning horizon. We propose and analyze a bandit algorithm for this problem. Since the bandit algorithm relies on solving an optimal task assignment problem repeatedly, we analyze the achievable regret in two cases. If we can solve the optimal task assignment exactly, we show that the regret for our algorithm is the same as that in the single-agent bandit task assignment algorithm. If, on the other hand, the exact solution is not accessible, we analyze the regret with respect to an oracle-based algorithm that uses a sub-optimal solver.**

I. Introduction

A multi-agent task assignment problem aims to maximize the reward from task completion by mapping tasks to agents, who may have heterogeneous capabilities. The main challenge in this problem is computational efficiency as the number of tasks and agents increase, particularly if agents have diverse preferences in terms of the reward they obtain from completing a particular task or characteristics in terms of the time they spend on a task.

Existing literature has studied heterogeneous multi-agent task assignment problems with uncertain rewards, while only a few works have considered uncertainty in task completion times. Most literature casts the task assignment problem while learning the rewards as a multi-armed bandit problem [1]. [2] presents a bandit algorithm to select the motor task that yields the best performance for an asynchronous brain-controlled button; [3] considers the distributed multi-user task assignment that takes user preference into account. However, these studies above do not consider the task completion time. Blocking bandits, a multi-armed bandit problem with stochastic rewards but known blocking times for task completion [4] has learned to enhance the expected cumulative rewards over time, but the task completion time is a fixed, known parameter, as opposed to our problem. Some studies in task assignment for heterogeneous crowdsourcing have also explored the algorithms aiming at maximal utilities, worker reliability, or preferences within a limited time or budget. Among them, [5], [6] have developed bandit-based task assignment algorithms focusing on task selection and worker selection, respectively; [7] uses bi-objective optimization with combinatorial multi-armed bandits; [8] uses a contextual bandit algorithm to assign workers to tasks based on their observed task acceptance behavior; [9] estimates workers' abilities and assigns them to tasks through an arm-limited, budget limited multi-armed bandit; [10] explores budget-limited task assignment problem with heterogeneous workers in both costs and the quality of the work; [11], [12] assign tasks to the workers who are more interested in the tasks with higher priorities. However, these crowdsourcing problems fail to consider the uncertain completion time as stochastic intervals. The problem setup in [13] takes both the stochastic reward and completion time into account, but their algorithm is not scalable when task number and team size grow, while [14] studies a scalable task assignment for heterogeneous multi-robot teams, but does not consider the uncertainties in task outcomes or completion times.

We study a task assignment problem in a heterogeneous multi-agent setting, where the members have different task completion times and generate different rewards for completing the same task. A central controller assigns tasks to a suitable agent subject to individual resource constraints on the tasks. The goal of the controller is to maximize the expected total reward all agents can receive over a given planning horizon. The reward and completion time for each member for any task follow unknown distributions. Consequently, we study the problem in a bandit framework to learn the optimal task assignment over the horizon.

Our main contributions are as follows. First, we formulate a multi-agent, heterogeneous task assignment problem in a bandit framework and propose an efficient task assignment algorithm that scales well with the number of agents and tasks. This algorithm adapts to problems of small sizes where we want to obtain an exact optimal task assignment, and problems with numerous agents and tasks where an optimal task-assignment algorithm is implausible, and thus a sub-optimal task assignment is necessary because of computational complexity. Next, we analyze the regret for our task-assignment algorithm. If we assume that the optimal task assignment algorithm is accessible in each round, we show that the regret for our algorithm is the same as that in the single-agent bandit task assignment algorithm. On the other hand, when the optimal solution is not accessible and we

Q. Wei, and V. Gupta are with the Elmore Family School of Electrical and Computer Engineering, Purdue University, West Lafayette, IN 47907, USA (emails: wei473@purdue.edu, gupta869@purdue.edu). V. Srivastava is with the Electrical and Computer Engineering, Michigan State University, East Lansing, MI 48824 (email: vaibhav@egr.msu.edu). The work was supported by xxx under Grant xxx.

follow a sub-optimal solution, we analyze the regret for our algorithm with respect to this sub-optimal algorithm. In either case, we show that the regret can be upper bounded as $O(\ln T)$, where T is the time horizon, demonstrating the effectiveness of the task assignment algorithm. The main challenge is in the sub-optimal regret analysis, as the task assignments at all rounds are based on the fact that we may only obtain a sub-optimal solution to a task assignment problem. We validate the proposed algorithm through a case study with three teams of different sizes, showing its effectiveness in practical applications.

The paper closest to ours is [13], which studies a task assignment problem with a single agent. As compared to that work, we consider a multi-agent setup where heterogeneity among various agents arises naturally. Second, while the algorithm in [13] is not scalable in our task-assignment setup, we extend it to an approximate algorithm that suits large-scale heterogeneous bandit task assignment problems. Third, we establish the performance analysis specifically for this approximate algorithm to show its effectiveness.

Notation: The (i,j)-th entry of a matrix M is denoted by M_{ij}. For two vectors a, b (resp. matrices A, B) of the same size, $a \leq b$ (resp. $A \leq B$) means that each entry of a (resp. A) is less or equal than the corresponding entry of b (resp. B). Given two n by m matrices A, B, we let $C = A \odot B$ represent the dot product of two matrices, i.e., $C_{ij} = A_{ij}B_{ij}$. Further, we define the function $\Sigma(A)$ that sums up all entries in the matrix A. We let $\mathbf{1}_N, \mathbf{0}_N$ represent the ones vector and zeros vector with length N, and let $\mathbf{I}_N$ represent the $N \times N$ identity matrix.

II. Problem Formulation

Consider a team consisting of several different agents or members that need to execute a set of tasks under resource constraints. These constraints can correspond to quantities such as energy or attention. After completion of each task, the team receives a reward. Both the constraints and the completion times can vary among the agents for a given task and for the same agents among various tasks. All tasks belong to a fixed set, but each task is ready to be assigned anew after completion. A central planner assigns the tasks to team members when both tasks and team members become available. The aim is to maximize the expected total team reward over a given horizon.

More formally, we denote the set of all tasks by $\mathcal{V} = (v_1, v_2, \ldots, v_N)$, where N is the total number of tasks. Similarly, denote the set of all agents by $\mathcal{M} = \{m_1, m_2, \ldots, m_M\}$, where M is the total number of agents or team members. We let the index set $I_\mathcal{V} = \{1, 2, \ldots, N\}$, $I_\mathcal{M} = \{1, 2, \ldots, M\}$, and $I_\mathcal{A} = \{(i,m) \mid m \in I_\mathcal{M}, i \in I_\mathcal{V}\}$. Each member $m \in \mathcal{M}$ has a constraint of $L_m \geq 0$ available resources (e.g. energy or attention), where it needs to allocate f_{im} amount of resources during the time the member executes the task v_i, for all $i \in I_\mathcal{V}$. We let L represent the set of all L_m.

A *task assignment* is a binary matrix $a \in \{0,1\}^{N\times M}$ whose (i,j)-th entry denotes whether the task i has been assigned to agent m (if $a_{ij} = 1$) or not (if $a_{ij} = 0$). Note that the task assignment may change over time. The task assignment should be feasible as defined below.

Definition 1 (Feasible Task Assignment)**.** *A task assignment a is* feasible *if it satisfies the following two properties:*

1) *For all $m \in I_\mathcal{M}$, the agent m can simultaneously execute the set of tasks assigned to her $\mathcal{V}_m = \{i \in I_\mathcal{V} \mid a_{im} = 1\}$ so that $\sum_{i\in I_\mathcal{V}} f_{im}a_{im} \leq L_m$.*
2) *Each task is assigned to at most one agent at the same time. In other words, $\sum_{m\in I_\mathcal{M}} a_{im} \leq 1$.*

Denote the set of all feasible task assignments as $\mathcal{A} \subseteq \{0,1\}^{N\times M}$. Similar to [13], we assume that $\mathcal{A}$ is closed under inclusion, that is, if $a \in \mathcal{A}$ and $b \leq a$, then $b \in \mathcal{A}$. We denote the maximum number of tasks the team can execute simultaneously as $\overline{L} = \max_{a\in\mathcal{A}} \sum_{(i,m)\in I_\mathcal{A}} a_{im}$.

We solve the problem over a given time horizon $t = 1, \cdots, T \in \mathbb{N}$. At each round t, the planner chooses a feasible task assignment $a_t \in \mathcal{A}$. For the binary matrix a_t, we denote the (i,m)-th entry as $a_{t,im}$, so that if $a_{t,im} = 1$, then member m will execute task i at time t. The member spends time $c_{t,im} \in \{C_l, C_l+1, \cdots, C_u\}$, where $C_l, C_u \in \mathbb{N}$ and $1 \leq C_l \leq C_u$ for a task assigned to it at round t. Thus, the member m will complete task i at time $t + c_{t,im}$. Upon completion, the member (equivalently, the team or the planner) will receive a reward $r_{t,im} \in [0,1]$. Moreover, the task i will be available for reassignment. In general, for all $m \in I_\mathcal{M}$, $\{r_{t,im}, c_{t,im}\}_{i\in I_\mathcal{V}}$ are drawn in an independently and identically manner from a distribution over $[0,1]^N \times \{C_l, C_l+1, \cdots, C_u\}^N$ which is independent of the time t. Note that $c_{t,im}$ and $r_{t,im}$ may be dependent. We denote the matrix $c_t \in \{C_l, C_l+1, \cdots, C_u\}^{N\times M}$ as the completion-time matrix, with the (i,m)-th entry defined as $c_{t,im}$; we denote the matrix $r_t \in [0,1]^{N\times M}$ as the reward matrix, with the (i,m)-th entry defined as $r_{t,im}$.

Note that at any time t, assignment of any new tasks must respect the constraints imposed by all tasks already assigned but uncompleted. Define the set of all assignments corresponding to uncompleted tasks at time t by the binary matrix $b_t \in \mathcal{A}$ and let $b_{t,im}$ representing the (i,m)-th entry of the binary matrix b_t. We define the set of available task assignments at time t by $\mathcal{A}_t$ through the relations

$$\begin{aligned} b_{t,im} &= \sum_{s=1}^{t-1} a_{s,im} \mathbb{1}[s + c_{s,im} > t], \\ \mathcal{A}_t &= \{a \in \mathcal{A} \mid a + b_t \in \mathcal{A}\}. \end{aligned} \tag{1}$$

Example 1. *In Fig. 1 we show a feasible task assignment for a team with 2 members $\mathcal{M} = \{m_1, m_2\}$ and 3 tasks $\mathcal{V} = \{v_1, v_2, v_3\}$, where $L_1 = 2, L_2 = 1$, $f_{11} = 1.5$, $f_{21} = f_{22} = 0.5$, $f_{31} = f_{12} = 1$, and $f_{32} = 0.8$. The completion times $c_{1,11} = c_{5,31} = c_{6,22} = 3$, $c_{1,21} = 5$, $c_{8,11} = 2$, $c_{1,32} = 4$.*

The goal of the planner is to generate the task assignment policy that maximizes the expected accumulated reward of the team over the problem horizon, i.e., the reward function

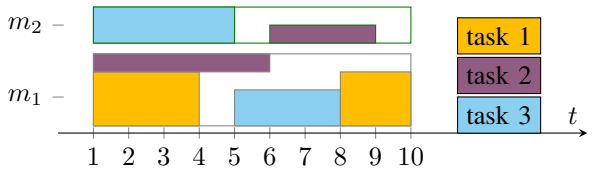


Fig. 1: A feasible task assignment in Example 1. The orange, purple, and blue filled rectangles represent task 1, 2, and 3, respectively. The width of the filled rectangle shows the task completion time, and the height shows how many resources the member needs to spend on the corresponding task. While the grey and green unfilled rectangles represent the total resources that members 1 and 2 have, respectively.

$\sum_{t=1}^{T} \Sigma(a_t \odot r_t)$. Specifically, a proper policy π of the planner is a (possibly stochastic) function that chooses feasible task assignment a_t that satisfies (1) based on information available at the planner prior to round t, but not on that after round t. Π is the set of all proper policies. We let the history h_t at round t consist of all prior task assignments $\{a_s\}_{s=1}^{t-1}$, and all observed rewards and completion times by the beginning of round t, $(r_{s,im}, c_{s,im})$ for all $(i,m) \in I_{\mathcal{A}}$ such that $s + c_{s,im} \leq t$. We denote $\pi = \{\pi_t\}_{t=1,2,\cdots}$ such that π_t maps from the history h_t to a feasible assignment $a_t \in \mathcal{A}$.

$$OPT = \max_{\pi^* \in \Pi} \mathbb{E}\left[\sum_{t=1}^{T} \Sigma(r_t \odot \pi_t^*(h_t^*))\right], \tag{2}$$

where $\{h_t^*\}_{t=1,2,\cdots}$ are the histories generated by π^*.

While this problem is already computationally complex, we study this problem under the assumption that the distributions specifying the reward $r_{t,im}$ and the task processing time $c_{t,im}$ are unknown. In this case, the planner needs to assign tasks to the agents to maximize the team reward and learn the distributions of rewards and processing times at the same time. To assess the performance of the task assigned by the planner, we use regret—defined as the difference between the optimal expected cumulative reward OPT and the expected cumulative reward that the planner's task assignment can achieve—with modification in the following.

To better assess our approximate task-assignment algorithm, we compare the expected cumulative reward of the team that the planner's task assignment can achieve with a sub-optimal benchmark, $\frac{1}{1+\alpha}OPT$, where $\alpha = \{0\} \cup [1, +\infty)$. Following the concept of $\frac{1}{1+\alpha}$-regret defined in blocking bandits [4] and $(\frac{1}{1+\alpha}, 1)$-regret defined in combinatorial bandits [15], we evaluate the performance of the learning algorithm that assign tasks to the team over time through the *α-regret* R_T^α defined as the difference between the expected accumulated rewards obtained with sub-optimal benchmark, $\frac{1}{1+\alpha}OPT$, and the expected accumulated rewards obtained by the proposed algorithm:

$$R_T^\alpha = \frac{1}{1+\alpha}OPT - \mathbb{E}\left[\sum_{t=1}^{T} \Sigma(r_t \odot a_t)\right], \tag{3}$$

We denote R_T^0 as the *optimal regret* and R_T^α as a *sub-optimal regret* for $\alpha \geq 1$, where optimal regret compares the expected accumulated rewards with the optimal benchmark OPT, while the sub-optimal regret compare that with a sub-optimal one.

Remark 1. *The heterogeneous bandit task assignment problem formulated above generalizes the stochastic combinatorial semi-bandits [16] and the multi-armed bandit problem with budget constraints [17].*

III. Task-Assignment Algorithm

In this section, we propose a learning algorithm for the problem posed above. The regret R_T^α is analytically cumbersome and we begin by upper bounding the regret. The following result can be proved along the linear of [13, Proposition 2.2] by generalizing the setting to a multi-player scenario.

We will find it useful to define the per-round reward function for agent m to complete task i as $q_{im} = \frac{\overline{r}_{im}}{\overline{c}_{im}}$, and we let $q \in [0,1]^{N \times M}$ with im-th entry equals to q_{im}. Define an optimal per-round task assignment as $a^* \in \operatorname{argmax}_{a \in \mathcal{A}} \Sigma(a \odot q)$.

Proposition 1. *Given any proper policy $\pi = \{\pi_t\}_{t=1,2,\cdots} \in \Pi$ and the sequence of histories $\{h_t\}_{t=1,2,\cdots}$ generated by using this policy, we can bound*

$$\mathbb{E}\left[\sum_{t=1}^{T} \sum_{m,i} r_{t,im} \pi_{im}(h_t))\right] \leq (T + C_u) \max_{a \in \mathcal{A}} \Sigma(q \odot a).$$

We have leave all proofs in the Appendix.

Given Proposition 1, we can bound R_T^α as follows:

$$\begin{aligned} R_T^\alpha &\leq \frac{1}{\alpha+1}(T + C_u) \max_{a \in \mathcal{A}} \Sigma(q \odot a) - \mathbb{E}\left[\sum_{t=1}^{T} \Sigma(r_t \odot a_t)\right] \\ &\leq \mathbb{E}\left[\sum_{t=1}^{T} \Sigma\Big(\frac{1}{\alpha+1} q \odot a^* - r_t \odot a_t\Big)\right] + \frac{C_u \overline{L}}{(\alpha+1) C_l}, \end{aligned} \tag{4}$$

Based on (4), we then build our learning algorithm 1 by estimating q.

Our learning algorithm builds on a phased-update method combined with a UCB based policy update. Specifically, we divide the time horizon into phases possibly of unequal length. The algorithm updates the task assignment policy at the beginning of each phase, but utilizes the same policy for the rest of the phase. While a similar phased-update method in a task assignment context (although for a single agent) was used in [13], we point out that the task assignment problem at the beginning of each phase is more complex than in [13] and we need to consider the scenario where the planner may not solve the problem optimally.

Our learning algorithm is presented in Algorithm 1. The first phase (Phase 0) is called the initialization phase. As stated in Line 1, the task assignment policy in this phase is that for B times, each member m complete each task i that satisfies $f_{im} \leq L_m$, where we let $B = \left\lceil 90 \frac{C_u}{C_l} \ln T \right\rceil$ and

Algorithm 1 Bandit Algorithm for Task Assigning

Input: C_l, C_u: lower and upper bounds for the processing time. B: length of initialization phase.

1: Initialization: For all $i \in I_{\mathcal{V}}$ and $m \in I_{\mathcal{M}}$ such that $f_{im} \leq L_m$, let member m execute the task i for B times. Record the completing round as t_1, and set $T_{im}(t_1) = B$.

2: **for** $s = 1, 2, \cdots$ **do**

3: Compute $\hat{q}_{im}(t_s)$ using (7).

4: Compute $l_s = C_l \min_{(i,m)\in I_{\mathcal{A}}} T_{im}(t_s) + 2C_u$, and set $t_{s+1} = t_s + l_s$.

5: Compute

$$a'_s \in \max_{a\in\mathcal{A}} \Sigma(\hat{q}(t_s) \odot a) \tag{5}$$

through a given Algorithm A.

6: **for** $t = t_s, t_s + 1, \cdots, t_{s+1} - 1$ **do**

7: Compute b_t using (1).

8: If $b_t \leq a'_s$, then $a_t \leftarrow a'_s - b_t$; else $a_t \leftarrow \{0\}^{N\times M}$.

9: **for** $i = 1, \cdots, N$ **do**

10: If member m completes task i at round t and we observe the corresponding reward completion time $r_{t',im}, c_{t',im}$, where t' is the task starting round, update $\bar{r}_{im}(t)$, $\bar{c}_{im}(t)$, $V^c_{im}(t)$. Set $T_{im}(t+1) = T_{im}(t) + 1$.

11: **end for**

12: **end for**

13: **end for**

$T > MNBC_u$. We can then upper bound the total regret in phase 0 by $\mathcal{O}(\frac{C_u MNB}{C_l}) = \mathcal{O}(\frac{C_u^2}{C_l^2} MN \ln T)$.

In every subsequent phase s prior to the phase that contains T, the lines 2–13 are followed. At the beginning of each phase, we compute the length l_s of that phase as

$$l_s = C_l \min_{(i,m)\in I_{\mathcal{A}}} T_{im}(t_s) + 2C_u. \tag{6}$$

We also set $t_s = \sum_{k=1}^{s-1} l_k$ as the time at which phase s begins. Algorithm 1 utilizes a UCB-type update. Thus, we maintain estimates $\hat{q}_{im}(t_s)$ of q_{im} through (7) below, where $\hat{r}_{t,im}$ and $\hat{c}_{t,im}$ are the empirical means of rewards and costs, $V^c_{t,im}$ are the empirical variance of costs, and $T_{im}(t)$ are the times that agent m has completed task i before time t.

$$\hat{q}_{im}(t) = \frac{\min\{1, \hat{r}_{t,im} + d^r_{t,im}\}}{\max\{C_l, \hat{c}_{t,im} + d^c_{t,im}\}}, \tag{7}$$

where $d^r_{t,im} = \sqrt{1.5 \ln t / T_{im}(t)}$ and $d^c_{t,im} = \sqrt{3V^c_{t,im} \ln t / T_{im}(t) + 9(C_u - C_l) \ln t / T_{im}(t)}$.

Having updated these estimates, we then calculate our task assignment policy for phase s using line 5. This step requires solving the linear optimization problem (7) via some given Algorithm A. During phase s, the members will follow the task assignment in ine 5 after completing all tasks that start in phase $s-1$. Note that each member m repeats the task i such that $a_{s,im} = 1$ once it is completed.

Remark 2 (Algorithm A)**.** *Algorithm 1 is agnostic to the algorithm A used to solve the problem in Line 5 and even whether the problem is solved exactly or approximately. The regret guarantees of the algorithm may be different for these cases and we will analyze them under two sets of assumptions. The ability to consider an approximate solution to* (5) *is particularly useful since* (5) *is equivalent to a mixed-integer optimization problem (MILP) equivalent to a generalized assignment problem (GAP), which is known to be NP-hard in general and computationally efficient algorithms to determine the optimal solution may not exist. Existing literature has proposed some algorithms to determine a sub-optimal solution. For instance, the algorithm in [18] guarantees an* $(1+\alpha)$-approximate *solution* a'*, with the guarantee that*

$$\Sigma(\hat{q}(t_s) \odot a') \geq \frac{1}{1+\alpha} \Sigma(\hat{q}(t_s) \odot a^*),$$

where a^* *is an optimal solution to* (5)*.*

We call Algorithm 1 an *exact algorithm* when the algorithm A in Line 5 will solve optimization problem (5) exactly. To obtain regret guarantees, we also consider the case when algorithm A only guarantees an $(1+\alpha)$-approximate solution to (5), such as the one in [18]. In this case, we call Algorithm 1 an $(1+\alpha)$*-approximate algorithm* or *approximate algorithm*.

IV. Theoretical results

In this section, we characterize the regret of the exact and approximate algorithms proposed above. Our proving methods follows the that in [13]. However, given that their results focus on single-agent task assignment problems with the premise that Algorithm 1 is an exact algorithm, our derivation of results remains challenging in the following two aspects. First, when the Algorithm 1 is an exact algorithm, we need to readapt the results in [13] into a heterogeneous multi-agent setup to provide an upper bound for the optimal regret. Second, when the Algorithm 1 is an $(1+\alpha)$-approximate algorithm, we need to derive the sub-optimal regret analysis with different techniques, as the task assignments at all rounds are based on that we only obtain a sub-optimal solution to (5).

As we set the initialization task completion times as $B = \left\lceil 90 \frac{C_u}{C_l} \ln T \right\rceil$, we have the properties (8)–(9) for all $(i,m) \in I_{\mathcal{A}}$, and Lemmas 1–2 below to derive the later analysis of regret bounds.

$$\begin{aligned} &T_{im}(t_s) \geq 90 \frac{C_u}{C_l} \ln T, \quad l_s \geq 90 C_u \ln T \\ &90 C_u \ln T \geq t_1 = \mathcal{O}\Big(\frac{C_u^2 N \ln T}{C_l}\Big). \end{aligned} \tag{8}$$

$$\begin{aligned} &\Pr[|\hat{r}_{im}(t) - \bar{r}_{im}| \geq d^r_{im}(t)] \leq \frac{2}{t^2} \\ &\Pr[|\hat{c}_{im}(t) - \bar{c}_{im}| \geq d^c_{im}(t)] \leq \frac{4}{t^2} \\ &\Pr[q_{im} \leq \hat{q}_{im}(t)] \leq 1 - \frac{6}{t^2}. \end{aligned} \tag{9}$$

Lemma 1. *For any $s \geq 1$ and $(i,m) \in A'_s$, the phase length $l_s \leq 2C_u(T_{im}(t_s+1)-T_{im}(t_s))$ and the accumulated execution times $T_{im}(t_{s+1}) \leq 4T_{im}(t_s)$.*

Lemma 2. *For any $s \geq 1$, there exists $(i,m) \in A'_s$, such that $T_{im}(t_{s+1}) \geq (1+\frac{C_l}{C_u})T_{im}(t_s)$. As a result, for all $s \geq 1$, $t_s \geq C_l(1+\frac{C_l}{C_u})^{\frac{s}{NM}-2}$.*

Lemma 2 and that $\ln(1+x) \geq x/2$ for any $x \in [0,1]$ infer that $\ln t_s \geq \ln C_l + (\frac{s}{NM}-2)\ln(1+\frac{C_l}{C_u}) > \ln C_l + (\frac{s}{NM}-2)\frac{C_l}{2C_u}$. And hence

$$S \leq NM\Big(\frac{2C_u}{C_l}\ln T + 2\Big) + 1 = \mathcal{O}(\frac{C_u}{C_l}NM\ln T)\,. \quad (10)$$

Further, we define an optimal per-round task assignment and its corresponding index set as $A^* = \{(i,m) \in I_{\mathcal{A}} \mid a^*_{im} = 1\}$. Further, we define the sub-optimality gaps as

$$\begin{aligned}
\Delta^\alpha_a &= \frac{1}{1+\alpha}\sum_{a^*_{im}=1} q_{im} - \sum_{a_{im}=1} q_{im} \quad \forall a \in \mathcal{A} \\
\Delta^\alpha_{im} &= \min_{a \in \mathcal{A} | a_{im}=1, \Delta^\alpha_a > 0} \Delta_a \quad \forall (m,i) \in I_{\mathcal{A}} \\
\underline{\Delta}^\alpha &= \begin{cases} \min_{(m,i)\in I_{\mathcal{A}}\setminus A^*} \Delta^0_{im} & \text{if } \alpha = 0 \\ \min_{(m,i)\in I_{\mathcal{A}}} \Delta^\alpha_{im} & \text{if } \alpha \in [1,+\infty)\,. \end{cases}
\end{aligned} \quad (11)$$

Finally, we denote the variance of the processing time $c_{t,im}$ as $\sigma^2_{im} = \mathbf{E}[(c_{t,im} - \bar{c}_{im})^2]$. We note that

$$\begin{aligned}
\sigma^2_{im} \leq \mathbf{E}[(c_{t,im}-C_l)^2] &\leq (C_u - C_l)\,\mathbf{E}[c_{t,im} - C_l] \\
&= (C_u - C_l)(\bar{c}_{im} - C_l)\,.
\end{aligned}$$

We then estimate the performance of the exact and approximate algorithms by analyzing the bounds for the optimal and sub-optimal regrets R^α_T where $\alpha \in \{0\} \cup [1,+\infty)$ defined as in (3).

Theorem 1. *Given that Algorithm 1 is an exact algorithm ($\alpha = 0$) or an $(1+\alpha)$-approximate algorithm ($\alpha \geq 1$), the regret R^α_T for Algorithm 1 is bounded above by*[1]

$$R^\alpha_T \leq \mathcal{O}\Big(\Big(\frac{1}{\underline{\Delta}^\alpha} + C_u\Big)\frac{C_u}{C_l^2}NM\overline{L}\ln T\Big)\,. \quad (12)$$

Further, for any distribution of completion times and rewards[2]*, the regret is bounded as $R^\alpha_T = \mathcal{O}\Big(\frac{1}{C_l}\sqrt{C_u NM\overline{L}T\ln T} + \frac{C_u^2}{C_l^2}NM\overline{L}\ln T\Big)$.*

Moreover, when Algorithm 1 is an exact algorithm, for any N, M, T, L such that $T = \Omega(C_u)$, there exists a problem instance for which the optimal regret of any task assignment algorithm is lower bounded by $R^0_T = \Omega(\frac{1}{C_l}\min\{\sqrt{C_u NM\overline{L}T}, \overline{L}T\})$.

Proof Sketch: We proof the results separately for the cases where Algorithm (1) is an exact algorithm and an approximate algorithm.

[1]Big O notation describes an upper bound on the growth rate of a function, and big Omega notation provides a lower bound.

[2]Gap-dependent regret (as in (12)) refers to regret bounds that explicitly depend on the specific parameters of the underlying distributions (e.g., sub-optimal gaps in (11)), while distribution-free regret here provides bounds that hold uniformly across all possible distributions.

If the Algorithm (1) is an exact algorithm, i.e., $\alpha = 0$. For this problem setting, we can consider the multi-agent problem as a generalization of task assignment for a single agent with NM tasks and maximal capacity $\overline{L}$, with extra constraints on the set of feasible actions to ensure that the planner will not assign the same task to multiple agents simultaneously. With this generalization, we can leverage the analysis from [13, Theorems 4.1 and 4.5] to obtain the upper and lower bounds on the optimal regret R^0_T.

When Algorithm (1) is an α-approximate algorithm with $\alpha \geq 1$, we have to revise the method in [13]. Given (4), we have

$$\begin{aligned}
R^\alpha_T &\leq \mathbb{E}\Big[\sum_{t=1}^{T}\Sigma\Big(\frac{1}{\alpha+1}q\odot a^* - r_t \odot a_t\Big)\Big] + \frac{C_u\overline{L}}{(\alpha+1)C_l} \\
&\leq \frac{\overline{L}}{(\alpha+1)C_l}\,\mathbb{E}[t_1] + R^{(1)}_T + R^{(2)}_T + \frac{C_u\overline{L}}{(\alpha+1)C_l}\,, \quad (13)
\end{aligned}$$

where

$$\begin{aligned}
R^{(1)}_T &= \mathbb{E}\Big[\sum_{s=1}^{S}\sum_{t=t_s}^{t_{s+1}-1}\Sigma\Big(\frac{1}{\alpha+1}q\odot a^* - q\odot a'_s\Big)\Big] \\
R^{(2)}_T &= \mathbb{E}\Big[\sum_{s=1}^{S}\sum_{t=t_s}^{t_{s+1}-1}\Sigma\Big(q\odot a'_s - r_t\odot a_t\Big)\Big]
\end{aligned}$$

and we define S as the phase that includes round T, $t_{S+1} = T+1$ for convenience. We then show that

$$R^{(1)}_T = \mathcal{O}\Big(\overline{L}\sum_{\forall(i,m)\in I_{\mathcal{A}}}\frac{\tilde{C}_{im}\ln T}{\Delta^\alpha_{im}}\ln T + \frac{C_u^2}{C_l^2}NM\overline{L}\Big) \quad (14)$$

$$R^{(2)}_T = \mathcal{O}\Big(\frac{C_u^2}{C_l^2}NM\overline{L}\ln T\Big)\,, \quad (15)$$

where $\tilde{C}_{im} \leq \mathcal{O}\Big(\frac{C_u}{\bar{c}_{im}C_l}\Big)$. Notice that (14) utilizes the fact that $\Sigma(\hat{q}(t_s)\odot a'_s) \geq \frac{1}{1+\alpha}\Sigma(\hat{q}(t_s)\odot a^*)$ in the proof. With (13)–(15), we then get the gap-dependent bound $R^\alpha_T \leq \mathcal{O}\big(\overline{L}\sum_{\forall m,i}\frac{\tilde{C}_{im}\ln T}{\Delta^\alpha_{im}} + \frac{C_u^2}{C_l^2}NM\overline{L}\ln T\big) \leq \mathcal{O}\big(\frac{\max_{\forall m,i}\tilde{C}_{im}}{\underline{\Delta}^\alpha}NM\overline{L}\ln T + \frac{C_u^2}{C_l^2}NM\overline{L}\ln T\big) \leq \mathcal{O}\big(\big(\frac{1}{\underline{\Delta}^\alpha} + C_u\big)\frac{C_u}{C_l^2}NM\overline{L}\ln T\big)$. Lastly, We obtain the distribution-independent regret bound by modifying the analysis of $R^{(1)}_T$. □

Remark 3. *In most cases, we can solve the MILP* (5) *and select the best task assignment problem only if the problem size is small (N, M are small values), given its NP-hardness.*

However, there exists a special case in which we can find an optimal solution to the MILP (5) *by solving just a linear program (LP). When $K_m f_{im} = L_m$ for some $K_m \in \mathbb{N}$ for all $m \in I_{\mathcal{M}}$, then the linear program below always has an integer solution.*

$$\begin{aligned}
\max_a \Big\{\Sigma(\hat{q}(t_s)\odot a) \mid &\sum_{i=1}^{N} f_{im}a_{im} \leq L_m\ \forall m \in I_{\mathcal{M}}, \\
&\sum_{m=1}^{M} a_{im} \leq 1 \quad \forall i \in I_{\mathcal{V}},\ a \in [0,1]^{N\times M}\Big\}
\end{aligned} \quad (16)$$

The result follows by first vectorizing a and recognizing that the resulting coefficient matrix is totally unimodular.

Therefore, we are able to obtain an integer optimal solution of the LP, and in fact, if we solve (16) *through the simplex algorithm, it will return an integer optimal solution, implying that we are able to solve the MILP problem* (5) *efficiently by solving a LP in this special case.*

V. Numerical Study

We demonstrate the proposed bandit heterogeneous task-assigning algorithm on a case study with 3 teams of different sizes. We perform the simulation using Matlab, where we use Gurobi with Yalmip toolbox to compute the exact solution for (5) at the beginning of each phase s, and use the algorithm proposed in [18]—described below in Algorithm 2—as Algorithm A in line (5) of Algorithm 1. Algorithm 2 will guarantee to find a 2-approximate solution to (5). Therefore, Algorithm 1 is an exact algorithm when directly computing (5) and a 2-approximate algorithm when using 2. We therefore simplify this 2-approximate algorithm as the approximate algorithm in the case study[3].

Algorithm 2 Approximate Task Assigning at Round t_s

Input: $\hat{q}_{im}(t_s)$ all $(i,m) \in I_{\mathcal{A}}$, L_m for all $m \in I_{\mathcal{M}}$
  for $s = 1, 2, \cdots$ **do**
    Initialization: let $Z \in \mathbb{N}^N$, and set $Z_i = -1$ for all $i \in I_{\mathcal{V}}$. Let $P \in \mathbb{R}^{N \times M}$.
    for $m = 1, \cdots, M$ **do**
      For $i \in I_{\mathcal{V}}$, set $P_{im} = \hat{q}_{im}(t_s)$ if $Z[i] = -1$, and set $P_{im} = \hat{q}_{im}(t_s) - \hat{q}_{iZ_i}(t_s)$ otherwise.
      Find $s^* \in \{\text{argmax}_s \sum_i P_{im} s_i \mid \sum_{i=1}^N s_i \leq L_m, s_i \in \{0,1\} \quad \forall i \in I_{\mathcal{V}}\}$
      For all i such that $s_i^* = 1$, set $Z_i = m$.
    end for
  end for
  For all $(i,m) \in I_{\mathcal{A}}$, $a'_{s,iZ_i} = 1$ if $Z_i \neq -1$

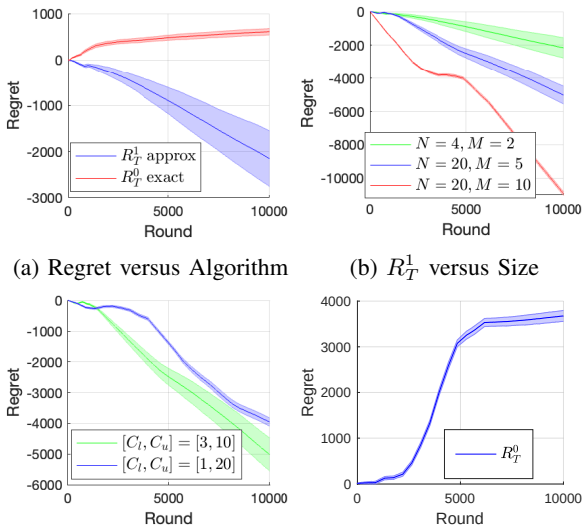
The small team has $N = 4$ tasks, $M = 2$ team members, $L = \{2, 1\}$, , $\bar{r}_{im} = 0.5$ for all i, m except that $r_{12} = 0.7$, $\bar{c}_{im} = 1.5$ for $i = 1, 2$, $m = 1, 2$, and $\bar{c}_{im} = 5$ for $i = 3, 4$, $m = 1, 2$, $f_{im} = 1$ for all i, m except that $f_{21} = 1.4$ and $f_{31} = 1.8$. The medium team has $N = 20$ tasks, $M = 5$ team members, and $L = \{2, 5, 4, 5, 6\}$. The large team has $N = 20$ tasks, $M = 10$ team members, and $L_m = 4$ for all m. We provide mean rewards and completion times and resource occupation for the medium and the large teams with code given the size of the problem. We set $T = 10000$ for all three teams.

The approximate algorithm can assign the tasks properly to the teams of all 3 sizes at each round in a timely manner, while the exact algorithm can only work for the smallest team. Fig. 2b demonstrates the cumulated sub-optimal regret R_T^1 of the 3 teams with different sizes for the approximate algorithm. For the smallest team, we also compute the cumulative optimal regret R_T^0 for the exact algorithm in comparison to R_T^1 for the approximate algorithm in Fig. 2a.

The cumulative optimal regret R_T^0 for the exact algorithm always scales logarithmically with T (the red curve in Fig.2a), while the sub-optimal regret R_T^1 for the approximate algorithm goes negative and scales linearly with time (the blue curve in Fig. 2a, all curves in Fig. 2b and Fig. 2c). The reason for these negative, linear sub-optimal regrets is that the approximate algorithm can outperform the benchmark $\frac{1}{2}OPT$, where OPT is the optimal accumulated reward that the team may obtain.

Fig. 2c, we are able to observe that the overall learning rate decreases when the lower and upper bounds become loose. Both the green and the blue curves represent the cumulative sub-optimal regret for the medium team. While keeping all other parameters the same, we change the completion bounds from $[C_l, C_u] = [3, 10]$ (green) to $[C_l, C_u] = [1, 20]$ (green), and the demonstrated degradation in performance is potentially caused by the large UCBs on q.

Finally, in Fig. 2d, we show that the large team has the sub-optimal regret R_T^1 for the approximate algorithm also scaling logarithmically with T, as the approximate algorithm can perform as good as the exact algorithm, depending on the detailed parameters.



(a) Regret versus Algorithm (b) R_T^1 versus Size

(c) R_T^1 versus $[C_l, C_u]$ (d) R_T^0 for largest team

Fig. 2: Cumulative regrets scale as logarithmic in Fig. 2a (exact method curve in red) and Fig. 2d, and negative linear regret in Fig. 2a, (exact method curve in red), Fig. 2c, and Fig. 2d.

VI. Conclusion

We introduced the heterogeneous bandit task assignment problem in a multi-agent setting and studied an efficient task assignment algorithm for both small-scale and large-scale problems where computing the exact optimal solution at each round can be intractable. We theoretically analyzed

[3]All our code and data used in the case study can be found in https://github.com/QinshuangCoolWei/Heterogeneous-Task-Assignment-with-Uncertain-Execution-Times-and-Preferences.git.

the optimal and sub-optimal regret bounds for both the exact and the approximate algorithms, and demonstrated the effectiveness of the algorithm via case studies with teams of varying sizes.

The proposed task assignment algorithm adapts to heterogeneous teammate preferences and abilities by modeling them as fixed distributions. However, in practice, these attributes can evolve over time, as individual proficiency and interest in tasks may fluctuate based on time, rewards, etc. Therefore, a future direction is to incorporate these evolving preferences and abilities into the task assignment process to plan more adaptively at all times.

REFERENCES

[1] R. Marjieh, A. Gokhale, F. Bullo, and L. Griffiths, "Task allocation in teams as a multi-armed bandit," *ACM Collective Intelligence*, 2024.

[2] J. Fruitet, A. Carpentier, R. Munos, and M. Clerc, "Automatic motor task selection via a bandit algorithm for a brain-controlled button," *Journal of neural engineering*, vol. 10, no. 1, p. 016012, 2013.

[3] X. Wang, J. Ye, and J. C. Lui, "Decentralized task offloading in edge computing: A multi-user multi-armed bandit approach," in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 1199–1208.

[4] S. Basu, R. Sen, S. Sanghavi, and S. Shakkottai, "Blocking bandits," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[5] H. Zhang, Y. Ma, and M. Sugiyama, "Bandit-based task assignment for heterogeneous crowdsourcing," *Neural computation*, vol. 27, no. 11, pp. 2447–2475, 2015.

[6] H. Zhang and M. Sugiyama, "Task selection for bandit-based task assignment in heterogeneous crowdsourcing," in *2015 Conference on Technologies and Applications of Artificial Intelligence (TAAI)*. IEEE, 2015, pp. 164–171.

[7] U. ul Hassan and E. Curry, "Efficient task assignment for spatial crowdsourcing: A combinatorial fractional optimization approach with semi-bandit learning," *Expert Systems with Applications*, vol. 58, pp. 36–56, 2016.

[8] U. U. Hassan and E. Curry, "A multi-armed bandit approach to online spatial task assignment," in *2014 IEEE 11th Intl Conf on Ubiquitous Intelligence and Computing and 2014 IEEE 11th Intl Conf on Autonomic and Trusted Computing and 2014 IEEE 14th Intl Conf on Scalable Computing and Communications and Its Associated Workshops*. IEEE, 2014, pp. 212–219.

[9] A. Rangi and M. Franceschetti, "Multi-armed bandit algorithms for crowdsourcing systems with online estimation of workers' ability." in *AAMAS*, 2018, pp. 1345–1352.

[10] L. Tran-Thanh, S. Stein, A. Rogers, and N. R. Jennings, "Efficient crowdsourcing of unknown experts using bounded multi-armed bandits," *Artificial Intelligence*, vol. 214, pp. 89–111, 2014.

[11] Y. Zhao, J. Xia, G. Liu, H. Su, D. Lian, S. Shang, and K. Zheng, "Preference-aware task assignment in spatial crowdsourcing," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 2629–2636.

[12] Y. Zhao, K. Zheng, H. Yin, G. Liu, J. Fang, and X. Zhou, "Preference-aware task assignment in spatial crowdsourcing: from individuals to groups," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 7, pp. 3461–3477, 2020.

[13] S. Ito, D. Hatano, H. Sumita, K. Takemura, T. Fukunaga, N. Kakimura, and K.-i. Kawarabayashi, "Bandit task assignment with unknown processing time," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[14] P. García, P. Caamaño, R. J. Duro, and F. Bellas, "Scalable task assignment for heterogeneous multi-robot teams," *International journal of advanced robotic systems*, vol. 10, no. 2, p. 105, 2013.

[15] N. Cesa-Bianchi and G. Lugosi, "Combinatorial bandits," *Journal of Computer and System Sciences*, vol. 78, no. 5, pp. 1404–1422, 2012.

[16] B. Kveton, Z. Wen, A. Ashkan, and C. Szepesvari, "Tight regret bounds for stochastic combinatorial semi-bandits," in *Artificial Intelligence and Statistics*. PMLR, 2015, pp. 535–543.

[17] Z. Dong, S. Das, P. Fowler, and C.-J. Ho, "Efficient nonmyopic online allocation of scarce reusable resources," in *AAMAS Conference proceedings*, 2021.

[18] R. Cohen, L. Katzir, and D. Raz, "An efficient approximation for the generalized assignment problem," *Information Processing Letters*, vol. 100, no. 4, pp. 162–166, 2006.

APPENDIX I
PROOF FOR PROPOSITION 1

Proof. Our multi-member task assignment setup with N tasks and M members with at most $\overline{L}$ simultaneous tasks is equivalent to that for a single agent task assignment with NM tasks and $\overline{L}$ maximal simultaneous tasks. As a result, we can directly apply the proof for [13, Proposition 2.2] to Proposition 1 here. □

APPENDIX II
PROOF FOR LEMMA 1

Proof. Given (8), we then have $\min_{(i,m)\in A'_s} T_{im}(t_s) \geq 2\frac{C_u}{C_l}$, and thus $l_s = C_l \min_{(i,m)\in I_\mathcal{A}} T_{im}(t_s) + 2C_u \geq 4C_u$. Therefore, for all $(i,m) \in A'_s$, the completion number is bounded from below $T_{im}(t_{s+1}) - T_{im}(t_s) \geq (l_s - 2C_u)/C_u$. Notice that $l_s - 2C_u - l_s/2 = ls/2 - 2C_u \geq 4C_u/2 - 2C_u \geq 0$. Hence $(l_s - 2C_u)/C_u \geq l_s/(2C_u)$, and we can conclude that $l_s \leq 2C_u(T_{im}(t_{s+1}) - T_{im}(t_s))$.

Similarly, the completion number is bounded from above $T_{im}(t_{s+1}) - T_{im}(t_s) \leq (l_s + 1)/C_l = (C_l \min_{(i,m)\in I_\mathcal{A}} T_{im}(t_s) + 2C_u + 1)/C_l = \min_{(i,m)\in I_\mathcal{A}} T_{im}(t_s) + 2C_u/C_l + 1/C_l \leq 3T_{im}(t_s)$, where the last inequality follows from that $2C_u > 1$ and that $\min_{(i,m)\in A'_s} T_{im}(t_s) \geq 2\frac{C_u}{C_l}$. □

APPENDIX III
PROOF FOR LEMMA 2

Proof. We let $(i^*, m^*) \in \operatorname{argmin}_{(i,m)\in A'_s} T_{im}(t_s)$. Then $l_s = T_{i^*,m^*}(t_s) + 2C_u$. The member m^* can complete the task i^* for at least $(l_s - 2C_u)/C_u = (C_l/C_u)T_{i^*,m^*}(t_s)$ times in phase s, and hence $T_{i^*,m^*}(t_{s+1}) \geq (1 + C_l/C_u)T_{i^*,m^*}(t_s)$. If $(i,m) \in A'_{s'}$ for some $s' < s$, then (8) indicates that $T_{im}(t_s) \geq \frac{C_u}{C_l}$. Then if for some $K \in \mathbb{N}$, we have $s \geq KNM$, then there exists $(i,m) \in I_\mathcal{A}$ such that $T_{im}(t_s) \geq (C_u/C_l)(1 + C_u/C_l)^{K-1}$ (which can be easily shown by contradiction). Since $t_s \geq C_l T_{im}(t_s)$, where right hand side is the shortest time for completion, for all $(i,m) \in I_\mathcal{A}$, we then have $t_s \geq C_l(\frac{C_u}{C_l})(1 + \frac{C_u}{C_l})^{\lfloor \frac{s}{NM} \rfloor - 1} \geq C_l(1 + \frac{C_u}{C_l})^{\frac{s}{NM} - 2}$.

□

APPENDIX IV
PROOF FOR THEOREM 1

In this section we assume that Algorithm A in line 5 of Algorithm 1 yields a $(1+\alpha)$-approximate solution to (5), and that Algorithm 1 is an $(1+\alpha)$-approximate algorithm.

We first denote the optimal UCB task assignment at round t_s as $\hat{a}^*_{s,im}$, i.e.,

$$\hat{a}_{s,im} \in \operatorname*{argmax}_{a\in\mathcal{A}} \Sigma(\hat{q}(t_s) \odot a) \tag{17}$$

We further define the index set for the optimal and chosen assignments

$$\begin{aligned} A'_s &= \{(i,m) \in I_\mathcal{A} \mid a'_{s,im} = 1\}, \\ \hat{A}_s &= \{(i,m) \in I_\mathcal{A} \mid \hat{a}_{s,im} = 1\}. \end{aligned} \tag{18}$$

Notice that, based on our assumption that Algorithm A yields a $(1+\alpha)$-approximate solution to (5), we have

$$\frac{1}{1+\alpha}\sum_{\hat{a}^*_{im}=1}\hat{q}_{im}(t_s) \le \sum_{a'_{im}=1}\hat{q}_{im}(t_s)\,. \tag{19}$$

For simplicity of expression, we define the quantity

$$d_{im}(t) = \sqrt{\frac{\tilde{C}_{im}}{\bar{c}_{im}}\frac{\ln t}{T_{im}(t)}}\,, \tag{20}$$

where

$$\begin{aligned}\tilde{C}_{im} = \frac{1}{\bar{c}_{im}}\Big(2\sqrt{1.5} + \frac{4}{\bar{c}_{im}}\Big(\sqrt{3\sigma^2_{im}} \\ + 15(C_u - C_l)\sqrt{\frac{C_l}{90C_u}}\Big)\Big)^2\,,\end{aligned} \tag{21}$$

Notice that

$$\begin{aligned}\tilde{C}_{im} &\le \mathcal{O}\Big(\frac{1}{\bar{c}_{im}}\Big(1+\frac{1}{\bar{c}^2_{im}}\Big(\sigma^2_{im} + \frac{(C_u-C_l)^2C_l}{C_u}\Big)\Big)\Big) \\ &\le \mathcal{O}\Big(\frac{1}{\bar{c}_{im}}\Big(1+\frac{C_u-C_l}{\bar{c}_{im}}\Big)\Big) \le \mathcal{O}\Big(\frac{C_u}{\bar{c}_{im}C_l}\Big)\,,\end{aligned} \tag{22}$$

We further define a designated set of phases

$$\mathcal{F}_s = \{s \ge 1 \mid \Delta^\alpha_{a'_s} \le \sum_{(i,m)\in A'_s} d_{im}(t_s), \Delta^\alpha_{a'_s} > 0\}\,, \tag{23}$$

and let $\overline{\mathcal{F}}_s$ be its complement set. Finally, we define the reward our algorithm can obtain from the phases in (23) as $\hat{R}_T$

$$\hat{R}_T = \sum_{s=1}^{S} l_s \Delta_{a'_s}\mathbb{1}[\mathcal{F}_s]\,. \tag{24}$$

Lemma 3. *For all phases $s \ge 1$, we have $\Delta_{a'_s} \le \sum_{(i,m)\in A'_s} d_i(t_s)$ with a probability at least $1-\frac{\overline{L}}{t_s^2}$. We then have $R_T^{(1)} \le \mathbb{E}[\hat{R}_T] + \mathcal{O}\big(\frac{MN\overline{L}}{C_l t_1}\big)$*

Proof. We first show $\Delta_{a'_s} \le \sum_{(i,m)\in A'_s} d_{i,m}(t_s)$ with a probability $1-\frac{6MN}{t_s^2}$ for all $s \ge 1$.

By revising the proof in [13, Lemma 4.3] from task $i \in I_{\mathcal{V}}$ to agent-task pair $(i,m) \in I_{\mathcal{A}}$, we first have (25)–(27) below.

$$\begin{aligned}\Pr[|\hat{r}_{im}(t) - \overline{r}_{im}| \ge d^r_{im}(t)] &\le \frac{2}{t^2} \\ \Pr[|\hat{c}_{im}(t) - \overline{c}_{im}| \ge d^c_{im}(t)] &\le \frac{4}{t^2} \\ \Pr[q_{im} \le \hat{q}_{im}(t)] &\le 1 - \frac{6}{t^2}\,,\end{aligned} \tag{25}$$

$$\hat{q}_{im}(t_s) \le q_{im}(t_s) + d_{im}(t_s)\,, \tag{26}$$

and that

$$\frac{l_s}{t_s^2} \le \frac{5}{t_1}\,. \tag{27}$$

We first note that

$$\Delta^\alpha_{a'_s} \le \frac{1}{1+\alpha}\sum_{a^*_{im}=1} q_{im} \le \frac{\overline{L}}{(1+\alpha)C_l} \tag{28}$$

as $q_{im} \le 1$, and there exists at most $\overline{L}$ pairs of (i,m) such that $a^*_{im} = 1$.

With probability at least $1-\frac{6NM}{t^2}$, we then have

$$\begin{aligned}\Delta^\alpha_{a'_s} &= \frac{1}{1+\alpha}\sum_{a^*_{im}=1} q_{im} - \sum_{a'_{im}=1} q_{im} \\ &\le \frac{1}{1+\alpha}\sum_{a^*_{im}=1}\hat{q}_{im}(t_s) - \sum_{a'_{im}=1} q_{im} &(29)\\ &\le \frac{1}{1+\alpha}\sum_{\hat{a}^*_{im}=1}\hat{q}_{im}(t_s) - \sum_{a'_{im}=1} q_{im} &(30)\\ &= \frac{1}{1+\alpha}\sum_{\hat{a}^*_{im}=1}\hat{q}_{im}(t_s) - \sum_{a'_{im}=1}\hat{q}_{im}(t_s) \\ &\qquad + \sum_{a'_{im}=1}(\hat{q}_{im}(t_s) - q_{im}(t_s)) \\ &\le \sum_{a'_{im}=1}\hat{q}_{im}(t_s) - \sum_{a'_{im}=1}\hat{q}_{im}(t_s) + \sum_{a'_{im}=1} d_{im}(t_s) &(31)\end{aligned}$$

where (29) is true with probability $1-\frac{6NM}{t^2}$ based on (25). (30) holds based on the optimum definition of $\hat{a}^*_{im} = 1$ in (17), and (31) follows immediately from (19).

Next, we rewrite and bound $R_T^{(1)}$ in the rest of the proof.

$$\begin{aligned}R_T^{(1)} &= \mathbb{E}\Big[\sum_{s=1}^{S}\sum_{t=t_s}^{t_{s+1}-1}\Sigma\Big(\frac{1}{\alpha+1}q\odot a^* - q\odot a'_s\Big)\Big] \\ &= \mathbb{E}\Big[\sum_{s=1}^{S} l_s\Delta^\alpha_{a'_s}\Big] \\ &= \mathbb{E}\Big[\sum_{s=1}^{S} l_s\Delta^\alpha_{a'_s}\mathbb{1}[\mathcal{F}_s] + \sum_{s=1}^{S} l_s\Delta^\alpha_{a'_s}\mathbb{1}[\overline{\mathcal{F}}_s]\Big] \\ &= \mathbb{E}[\hat{R}_T] + \mathbb{E}\Big[\sum_{s=1}^{S} l_s\Delta^\alpha_{a'_s}\mathbb{1}[\overline{\mathcal{F}}_s]\Big]\,,\end{aligned} \tag{32}$$

where

$$\begin{aligned}&\mathbb{E}\Big[\sum_{s=1}^{S} l_s\Delta^\alpha_{a'_s}\mathbb{1}[\overline{\mathcal{F}}_s]\Big] \\ &= \mathbb{E}\Big[\sum_{s=1}^{S} l_s\Delta^\alpha_{a'_s}\mathbb{1}[\Delta_{a'_s} > \sum_{(i,m)\in A'_s} d_{im}(t_s)]\Big] \\ &\qquad + \mathbb{E}\Big[\sum_{s=1}^{S} l_s\Delta^\alpha_{a'_s}\mathbb{1}[\Delta^\alpha_{a'_s} < 0]\Big] \\ &\le \mathbb{E}\Big[\sum_{s=1}^{S} l_s\frac{\overline{L}}{(1+\alpha)C_l}\frac{6MN}{t_s^2}\Big] = \frac{6MN\overline{L}}{(1+\alpha)C_l}\,\mathbb{E}\Big[\frac{l_s}{t_s^2}\Big] &(33)\\ &\le \frac{30MN\overline{L}}{(1+\alpha)C_l t_1}\,, &(34)\end{aligned}$$

where (33) follows from that (31) holds with probability at least $1-\frac{6NM}{t^2}$ and (28), and (34) follows from (27).

Then (32) and (34) together yields the bound on $R_T^{(1)}$ in Lemma 3. □

To analyze the upper bound of $\mathbb{E}[\hat{R}_T]$, we first obtain Lemma 4 by directly applying the proof for [13, Lemma A.4]

to the lemma by revising the task i in its proof to agent-task pair $(i,m) \in I_{\mathcal{A}}$.

Lemma 4. *For all $s \leq 1$ and $(i,m) \in A'_s$, we have*

$$\mathbb{E}[T_{im}(t_{s+1}) - T_{im}(t_s) \mid l_s] \geq \frac{l_s - 2C_u}{\overline{c}_{im}}. \tag{35}$$

Therefore, we have

$$l_s \leq 2\overline{c}_{im}\,\mathbb{E}[T_{im}(t_{s+1}) - T_{im}(t_s) \mid l_s, T_{im}(t_s)]. \tag{36}$$

We now bound $\mathbb{E}[\hat{R}_T]$ used in the upper bound of $R_T^{(1)}$ in Lemma 3.

Lemma 5. $\mathbb{E}[\hat{R}_T] = \mathcal{O}\Big(\overline{L}\sum_{(i,m)\in A'_s} \frac{\tilde{C}_{im}\ln T}{\Delta^{\alpha}_{im}}\Big)$

Proof. By leveraging [13, Lemma A.2] with $\{\alpha_k\}$ and $\{\beta_k\}$ such that $\sum_{k=1}^{\infty}\frac{\alpha_k}{\beta_k} = \mathcal{O}(1)$, if $\mathcal{F}_s$ defined in (23) occurs, we then have $\sum_{(i,m)\in A'_s}\mathbb{1}[(i,m), \Delta^{\alpha}_{a'_s} \leq \overline{L}\sqrt{\alpha_k}d_{im}(t_s)] \geq \beta_k$ for some k. Therefore,

$$\mathbb{1}[\mathcal{F}_s] \leq \sum_{k=1}^{\infty}\frac{1}{\overline{L}\beta_k}\sum_{(i,m)\in A'_s}\mathbb{1}[\mathcal{G}_{s,k,im}] \tag{37}$$

where $G_{s,k,im} = \{(i,m) \in A'_s \mid \Delta_{a'_s} \leq \sqrt{\alpha_k}d_{im}(t_s)\}$.

We then bound $\mathbb{E}[\hat{R}_T]$ as below

$$\begin{aligned}
\mathbb{E}[\hat{R}_T] &= \mathbb{E}\Big[\sum_{s=1}^{S} l_s\Delta^{\alpha}_{a'_s}\mathbb{1}[\mathcal{F}_s]\Big] \\
&\leq \sum_{(i,m)\in A'_s}\sum_{k=1}^{\infty}\frac{1}{\overline{L}\beta_k}\sum_{s=1}^{S} l_s\Delta^{\alpha}_{a'_s}\mathbb{1}[\mathcal{G}_{s,k,im}] \\
&\leq \sum_{(i,m)\in A'_s}\sum_{k=1}^{\infty}\frac{1}{\beta_k}\sum_{s=1}^{S} l_s\sqrt{\alpha_k}d_{im}(t_s)\mathbb{1}[\mathcal{G}_{s,k,im}] \\
&= \sum_{(i,m)\in A'_s}\sum_{k=1}^{\infty}\frac{\sqrt{\alpha_k}}{\beta_k}\sum_{s=1}^{S} l_s\sqrt{\frac{\tilde{C}_{im}}{\overline{c}_{im}}\frac{\ln t_s}{T_{im}(t_s)}}\mathbb{1}[\mathcal{G}_{s,k,im}] \\
&\leq \sqrt{\ln T}\sum_{(i,m)\in A'_s}\sqrt{\frac{\tilde{C}_{im}}{\overline{c}_{im}}}\sum_{k=1}^{\infty}\frac{\sqrt{\alpha_k}}{\beta_k}\sum_{s=1}^{S}\frac{l_s\mathbb{1}[\mathcal{G}_{s,k,im}]}{\sqrt{T_{im}(t_s)}}.
\end{aligned} \tag{38}$$

With given k and i, we then bound the term $\sum_{s=1}^{S}\frac{l_s\mathbb{1}[\mathcal{G}_{s,k,im}]}{\sqrt{T_{im}(t)}}$ in (38). Suppose that $\mathcal{G}_{s,k,im}$ occurs, we then have $\Delta^{\alpha}_{im} \leq \Delta^{\alpha}_{a'_s} \leq \overline{L}\sqrt{\alpha_k}d_{im}(t_s) \leq \overline{L}\sqrt{\frac{\alpha_k\tilde{C}_{im}\ln T}{\overline{c}_{im}T_{im}(t_s)}}$, where the first inequality follows from the definition of Δ^{α}_{im}. As a result, $\sqrt{T_{im}(t_s)} \leq \frac{\overline{L}}{\Delta^{\alpha}_{im}}\sqrt{\frac{\alpha_k\tilde{C}_{im}\ln T}{\overline{c}_{im}}}$, and we define it as γ. Based on this a sufficient but not necessary condition, we can derive that

$$\sum_{s=1}^{S}\frac{l_s\mathbb{1}[\mathcal{G}_{s,k,im}]}{\sqrt{T_{im}(t_s)}} \leq \sum_{s=1}^{S}\frac{l_s\mathbb{1}[(i,m)\in A'_s]\mathbb{1}[\sqrt{T_{im}(t_s)} \leq \gamma]}{\sqrt{T_{im}(t_s)}}. \tag{39}$$

Using (36) in Lemma 4, we then have

$$\begin{aligned}
&\mathbb{E}\Big[\frac{l_s\mathbb{1}[(i,m)\in A'_s]}{\sqrt{T_{im}(t_s)}}\Big] \\
&\leq \mathbb{E}\Big[\frac{2\overline{c}_{im}\,\mathbb{E}[T_{im}(t_{s+1}) - T_{im}(t_s) \mid l_s, T_{im}(t_s)]}{\sqrt{T_{im}(t_s)}}\Big] \\
&= 2\overline{c}_{im}\sum_{\substack{\forall\ l_s \\ T_{im}(t_s)}} Pr\{l_s, T_{im}(t_s)\}\frac{1}{\sqrt{T_{im}(t_s)}}\cdot \\
&\quad\sum_{T_{im}(t_{s+1})} Pr\{T_{im}(t_{s+1}) \mid l_s, T_{im}(t_s)\}(T_{im}(t_{s+1}) - T_{im}(t_s)) \\
&= 2\overline{c}_{im}\sum_{\substack{l_s, T_{im}(t_s), \\ T_{im}(t_{s+1})}} Pr\{l_s, T_{im}(t_s), T_{im}(t_{s+1})\}\cdot \\
&\qquad\qquad\qquad\qquad\qquad\qquad\frac{T_{im}(t_{s+1}) - T_{im}(t_s)}{\sqrt{T_{im}(t_s)}} \\
&= 2\overline{c}_{im}\,\mathbb{E}\Big[\frac{T_{im}(t_{s+1}) - T_{im}(t_s)}{\sqrt{T_{im}(t_s)}}\Big],
\end{aligned} \tag{40}$$

where we have

$$\begin{aligned}
\frac{T_{im}(t_{s+1}) - T_{im}(t_s)}{\sqrt{T_{im}(t_s)}} &= 3\frac{T_{im}(t_{s+1}) - T_{im}(t_s)}{\sqrt{4T_{im}(t_s)} + \sqrt{T_{im}(t_s)}} \\
&\leq 3\frac{T_{im}(t_{s+1}) - T_{im}(t_s)}{\sqrt{T_{im}(t_{s+1})} + \sqrt{T_{im}(t_s)}} \\
&= 3\sqrt{T_{im}(t_{s+1}) - T_{im}(t_s)},
\end{aligned} \tag{41}$$

and the inequality in (41) follows from Lemma.

Given (39)–(41) and define s_0 as the phase such that $T_{im}(t_{s_0+1} > \gamma$ while $T_{im}(t_{s_0} \leq \gamma$, we then have

$$\begin{aligned}
&\sum_{s=1}^{S}\frac{l_s\mathbb{1}[\mathcal{G}_{s,k,im}]}{\sqrt{T_{im}(t_s)}} \\
&\leq 2\overline{c}_{im}\,\mathbb{E}\Big[\sum_{s=1}^{S}\frac{T_{im}(t_{s+1}) - T_{im}(t_s)}{\sqrt{T_{im}(t_s)}}\mathbb{1}[\sqrt{T_{im}(t_s)} \leq \gamma]\Big] \\
&\leq 6\overline{c}_{im}\,\mathbb{E}\Big[\sum_{s=1}^{S}\sqrt{T_{im}(t_{s+1}) - T_{im}(t_s)}\mathbb{1}[\sqrt{T_{im}(t_s)} \leq \gamma]\Big] \\
&=\leq 6\overline{c}_{im}\,\mathbb{E}\Big[\sum_{s=1}^{s_0}\sqrt{T_{im}(t_{s+1}) - T_{im}(t_s)}\mathbb{1}[\sqrt{T_{im}(t_s)} \leq \gamma]\Big] \\
&= 6\overline{c}_{im}T_{im}(t_{s_0+1}) \leq 12\overline{c}_{im}T_{im}(t_{s_0+1}) \leq 12\overline{c}_{im}\gamma \\
&= \frac{12\overline{L}\overline{c}_{im}}{\Delta^{\alpha}_{im}}\sqrt{\frac{\alpha_k\tilde{C}_{im}\ln T}{\overline{c}_{im}}} = \frac{12\overline{L}}{\Delta^{\alpha}_{im}}\sqrt{\alpha_k\overline{c}_{im}\tilde{C}_{im}\ln T}.
\end{aligned}$$

Then, from (38) and above, we conclude that

$$\begin{aligned}
\mathbb{E}[\hat{R}_T] &\leq \sqrt{\ln T}\sum_{(i,m)\in A'_s}\sqrt{\frac{\tilde{C}_{im}}{\overline{c}_{im}}}\sum_{k=1}^{\infty}\frac{\sqrt{\alpha_k}}{\beta_k}\sum_{s=1}^{S}\frac{l_s\mathbb{1}[\mathcal{G}_{s,k,im}]}{\sqrt{T_{im}(t_s)}} \\
&\leq \sqrt{\ln T}\sum_{(i,m)\in A'_s}\sqrt{\frac{\tilde{C}_{im}}{\overline{c}_{im}}}\sum_{k=1}^{\infty}\frac{\sqrt{\alpha_k}}{\beta_k}\frac{12\overline{L}}{\Delta^{\alpha}_{im}}\sqrt{\alpha_k\overline{c}_{im}\tilde{C}_{im}\ln T} \\
&= \mathcal{O}\Big(\overline{L}\ln T\sum_{(i,m)\in A'_s}\frac{\tilde{C}_{im}}{\Delta^{\alpha}_{im}}\frac{\sqrt{\alpha_k}}{\beta_k}\Big) = \mathcal{O}\Big(\overline{L}\sum_{(i,m)\in A'_s}\frac{\tilde{C}_{im}\ln T}{\Delta^{\alpha}_{im}}\Big).
\end{aligned}$$

□

Lemma 6. $R_T^{(2)} = \mathbb{E}\Big[\sum_{s=1}^{S}\sum_{t=t_s}^{t_{s+1}-1}\Sigma\Big(q\odot a'_s - r_t\odot a_t\Big)\Big] = \mathcal{O}\Big(\frac{C_u^2}{C_l^2}NM\overline{L}\ln T\Big)$

Proof. First, following the proof of [13, Lemma A.3], we can derive that

$$\mathbb{E}\Big[\sum_{t=t_s}^{t_{s+1}-1}\Sigma\Big(q\odot a'_s - r_t\odot a_t\Big)\Big] \le 3\overline{L}\frac{C_u}{C_l}, \qquad (42)$$

given any a'_s, t_s, and l_s. For clarity, we restate the proof in our problem setting in the following.

Given $s \ge 1$, for any task-member pair $(i,m) \in I_{\mathcal{A}}$, the number of times that the member m will start the task i during the phase s is at least $T_{im}(t_{s+1}) - T_{im}(t_s) - 1$. Therefore, we have

$$\begin{aligned}
&\mathbb{E}\Big[\sum_{t=t_s}^{t_{s+1}-1}\Sigma\big(r_t\odot a_t\big)\Big]\\
&\ge \mathbb{E}\Big[\sum_{(i,m)\in A'_s}\overline{r}_{im}(T_{im}(t_{s+1}) - T_{im}(t_s) - 1)\Big]\\
&\ge \sum_{(i,m)\in A'_s}\overline{r}_{im}\Big[\sum_{\substack{l_s, T_{im}(t_s),\\ T_{im}(t_{s+1})}} Pr\{l_s, T_{im}(t_s), T_{im}(t_{s+1})\}\cdot\\
&\qquad\qquad (T_{im}(t_{s+1}) - T_{im}(t_s) - 1)\Big]\\
&\ge \sum_{(i,m)\in A'_s}\overline{r}_{im}\Big[\sum_{\substack{l_s,\\ T_{im}(t_s)}} Pr\{l_s\}\cdot\sum_{T_{im}(t_{s+1})}\\
&Pr\{T_{im}(t_{s+1}), T_{im}(t_s)) \mid l_s\}(T_{im}(t_{s+1}) - T_{im}(t_s) - 1)\Big]\\
&= \sum_{(i,m)\in A'_s}\overline{r}_{im}\,\mathbb{E}\Big[T_{im}(t_{s+1}) - T_{im}(t_s) - 1 \mid l_s\Big]\\
&\ge \sum_{(i,m)\in A'_s}\overline{r}_{im}\,\mathbb{E}\Big[\frac{l_s - 2C_u}{\overline{c}_{im}} - 1\Big] \qquad (43)\\
&= \sum_{(i,m)\in A'_s}\overline{r}_{im}\,\mathbb{E}\Big[\frac{l_s - 2C_u - \overline{c}_{im}}{\overline{c}_{im}}\Big]\\
&\ge \sum_{(i,m)\in A'_s}\overline{r}_{im}\,\mathbb{E}\Big[\frac{l_s - 3C_u}{\overline{c}_{im}}\Big] \qquad (44)\\
&= \sum_{(i,m)\in A'_s}\mathbb{E}\Big[q_{im}(l_s - 3C_u)\Big]\\
&\ge \sum_{(i,m)\in A'_s}\mathbb{E}\Big[q_{im}l_s\Big] - 3\frac{C_u}{C_l}\overline{L} \qquad (45)\\
&= \mathbb{E}\Big[l_s\Sigma(q_{im}\odot a'_s)\Big] - 3\frac{C_u}{C_l}\overline{L},
\end{aligned}$$

where the inequality (43) follows from Lemma 4, (44) follows from that $\overline{c}_{im} \le C_u$, (45) follows from that the team can execute at most $\overline{L}$ tasks at the same time, while $q_{im} \le 1/C_l$. We then directly obtain (42) and derive the upper bound for $R_T^{(2)}$.

$$\begin{aligned}
R_T^{(2)} &= \mathbb{E}\Big[\sum_{s=1}^{S}\sum_{t=t_s}^{t_{s+1}-1}\Sigma\Big(q\odot a'_s - r_t\odot a_t\Big)\Big]\\
&\le S\cdot 3\frac{C_u}{C_l}\overline{L} \le \mathcal{O}(\frac{C_u}{C_l}NM\ln T\cdot 3\frac{C_u}{C_l}\overline{L})\\
&= \mathcal{O}(\frac{C_u^2}{C_l^2}NM\overline{L}\ln T)\,.
\end{aligned}$$

where the last inequality that bounds the number of total phases S follows from (10). □

Given Lemmas 3, 5, and 6, we then cast bounds on R_T^{α} following $R_T^{(1)}$ and $R_T^{(2)}$. We show the proof for Theorem 1 as below.

Proof. When the Algorithm (1) is an exact algorithm, i.e., $\alpha = 0$. For this problem setting, we can consider the multi-agent problem as a generalization of task assignment for a single agent with NM tasks and maximal capacity $\overline{L}$, with extra constraints on the set of feasible actions to ensure that the planner will not assign the same task to multiple agents simultaneously. With this generalization, we can leverage the analysis from [13, Theorems 4.1 and 4.5] to obtain the upper and lower bounds on the optimal regret R_T^0.

When Algorithm (1) is an α-approximate algorithm with $\alpha \ge 1$, we have to revise the method in [13]. Given Lemmas 3 and 5, we have

$$\begin{aligned}
R_T^{(1)} &= \mathbb{E}\Big[\sum_{s=1}^{S}\sum_{t=t_s}^{t_{s+1}-1}\Sigma\Big(\frac{1}{\alpha+1}q\odot a^* - q\odot a'_s\Big)\Big]\\
&\le \mathbb{E}[\hat{R}_T] + \mathcal{O}\big(\frac{MN\overline{L}}{C_l t_1}\big)\\
&= \mathcal{O}\Big(\overline{L}\sum_{(i,m)\in A'_s}\frac{\tilde{C}_{im}\ln T}{\Delta_{im}^{\alpha}} + \frac{MN\overline{L}}{C_l t_1}\Big)\\
&= \mathcal{O}\Big(\overline{L}\sum_{(i,m)\in A'_s}\frac{\tilde{C}_{im}\ln T}{\Delta_{im}^{\alpha}} + \frac{C_u^2}{C_l^2}NM\overline{L}\Big) \qquad (46)
\end{aligned}$$

where we derive (46) based on Lemma 2, such that $t_1 \ge C_l(1+\frac{C_l}{C_u})^{\frac{1}{NM}-2} \ge C_l(1+\frac{C_l}{C_u})^{-2} \ge \frac{C_u^2}{C_l}$.

Combining this with the upper bound on $R_T^{(2)}$ from 6, we then get the gap-dependent bound

$$\begin{aligned}
R_T^{\alpha} &\le \mathcal{O}\big(\overline{L}\sum_{\forall m,i}\frac{\tilde{C}_{im}\ln T}{\Delta_{im}^{\alpha}} + \frac{C_u^2}{C_l^2}NM\overline{L}\ln T\big)\\
&\le \mathcal{O}\big(\frac{\max_{\forall m,i}\tilde{C}_{im}}{\underline{\Delta}^{\alpha}}NM\overline{L}\ln T + \frac{C_u^2}{C_l^2}NM\overline{L}\ln T\big)\\
&\le \mathcal{O}\Big(\Big(\frac{1}{\underline{\Delta}^{\alpha}} + C_u\Big)\frac{C_u}{C_l^2}NM\overline{L}\ln T\Big)\,.
\end{aligned}$$

Lastly, to obtain the distribution-independent regret bound, we modifying the analysis of $\hat{R}_T$. Given any $\epsilon > 0$, we can then bound $\hat{R}_T$ from above

$$\mathbb{E}[\hat{R}_T] = \mathbb{E}\Big[\sum_{s=1}^{S} l_s\Delta_{a'_s}^{\alpha}\mathbb{1}[\mathcal{F}_s]\Big]$$

$$
\begin{aligned}
&\leq \sum_{s=1}^{S} \mathbb{E}\left[l_s \Delta^{\alpha}_{a'_s} \mathbb{1}[\mathcal{F}_s, \Delta^{\alpha}_{a'_s} > \epsilon] + l_s \Delta^{\alpha}_{a'_s} \mathbb{1}[\mathcal{F}_s, \Delta^{\alpha}_{a'_s} \leq \epsilon] \right] \\
&\leq \mathcal{O}\Big(\overline{L} \sum_{(i,m)\in A'_s} \frac{\tilde{C}_{im} \ln T}{\Delta^{\alpha}_{im}} \mathbb{1}[\Delta^{\alpha}_{a'_s} \leq \epsilon] \Big] + \epsilon T \Big) \qquad (47) \\
&\leq \mathcal{O}\Big(\overline{L} \sum_{(i,m)\in A'_s} \frac{\tilde{C}_{im} \ln T}{\epsilon} + \epsilon T \Big)
\end{aligned}
$$

where the first part of (47) follows exactly from the proof of Lemma 5, and the latter part of (47) follows from that $l_s < T$. Then by setting $\epsilon = \sqrt{\frac{C_u MN\overline{L} \ln T}{C_l^2 T}}$, we obtain that $\mathbb{E}[\hat{R}_T] \leq \mathcal{O}\Big(\frac{1}{C_l} \sqrt{C_u MN\overline{L} T \ln T} \Big)$. And we then get the gap-free bound for R_T^{α} as in Theorem 1 that

$$
R_T^{\alpha} \leq \mathcal{O}\Big(\Big(\frac{1}{\underline{\Delta}^{\alpha}} + C_u \Big) \frac{C_u}{C_l^2} NM\overline{L} \ln T \Big) .
$$

□

APPENDIX V
PROOF OF REMARK 3

Proof. We vectorize a as $\vec{a}$ with length NM such that $\vec{a}_{iN+m} = a_{im}$ and show the results through the property of TUM. One can verify that we can rewrite the constraints in (16) as $\begin{bmatrix} E^{\top} & \mathbf{I}_N & -\mathbf{I}_N \end{bmatrix}^{\top} \vec{a} \leq \begin{bmatrix} \vec{b}^{\top} & \mathbf{1}_{NM}^{\top} & \mathbf{0}_{NM}^{\top} \end{bmatrix}^{\top}$, where E is a $(M+N) \times MN$ matrix and $\vec{b}$ is a vector of length $(M+N)$:

$$
E = \begin{bmatrix} \mathbf{1}_N^{\top} & \mathbf{0}_N^{\top} & \cdots & \mathbf{0}_N^{\top} \\ \mathbf{0}_N^{\top} & \mathbf{1}_N^{\top} & \cdots & \mathbf{0}_N^{\top} \\ \vdots & \ddots & \ddots & \vdots \\ \mathbf{0}_N^{\top} & \mathbf{0}_N^{\top} & \cdots & \mathbf{1}_N^{\top} \\ \mathbf{I}_N & \mathbf{I}_N & \cdots & \mathbf{I}_N \end{bmatrix} \text{ and } \vec{b} = \begin{bmatrix} K_1 \\ K_2 \\ \vdots \\ K_M \\ \mathbf{1}_N \end{bmatrix} .
$$

By Hoffman's sufficient conditions, E is a TUM, and by the property that concatenation of TUM with identity matrix and changing sign preserve TUM, we know that $\begin{bmatrix} E^{\top} & \mathbf{I}_N & -\mathbf{I}_N \end{bmatrix}^{\top}$ is a TUM. The proposition then follows directly from the property of TUM. □